

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально-науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 2

з навчальної дисципліни

"Технології розробки корпоративних Web-додатків"

Виконав:

Студент 402 – ТН

Ткаченко Вадим Олександрович

Перевірив:

Тютюнник Петро Богданович

Лабораторна робота №2

Проектування архітектури програмного забезпечення та моделювання даних

Завдання

1. Обґрунтування вибору технологічного стеку (Technology Stack) На основі вимог, зібраних у першій лабораторній роботі, оберіть технології для реалізації проєкту.

- Створіть документ або розділ у звіті, де чітко визначте:
 - Мова програмування Back-end: (наприклад, C#/.NET, Java/Spring, Node.js, Python/Django).
 - Фреймворк Front-end: (наприклад, React, Angular, Vue.js або Server-Side Rendering — Razor/Thymeleaf).
 - Система керування базами даних (СКБД): (PostgreSQL, MS SQL, MongoDB, Redis тощо).
 - Інструменти DevOps/Hosting: (Docker, GitHub Actions, Azure/AWS — опціонально).
- Надайте обґрунтування: Чому саме ці технології підходять для вирішення вашої бізнес-задачі? (Наприклад: "Обрано MongoDB, оскільки структура даних каталогу товарів є гнучкою та часто змінюється" або "Обрано .NET через надійну типізацію та швидкість розробки корпоративних API").

2. Архітектурне моделювання за методологією C4 (C4 Model) Спроектуйте архітектуру вашої системи, створивши три рівні діаграм C4. Це допоможе зрозуміти структуру системи від загального до конкретного.

- Рівень 1: System Context Diagram (Контекст).
 - Покажіть вашу систему як одну "чорну скриньку" в центрі.
 - Відобразіть усіх користувачів (Actors) та зовнішні системи (наприклад, платіжні шлюзи, API поштових розсилок), з якими вона взаємодіє.
- Рівень 2: Container Diagram (Контейнери).
 - "Відкрийте" систему і покажіть її основні технічні блоки (контейнери).

- Це можуть бути: Single Page Application, Mobile App, API Application, Database, File System.
- Вкажіть технології для кожного контейнера та протоколи взаємодії між ними (HTTP/HTTPS, TCP, GRPC).
- Рівень 3: Component Diagram (Компоненти).
 - Деталізуйте один основний контейнер (наприклад, API Application).
 - Покажіть внутрішні компоненти (наприклад: AuthController, EmailService, UserRepository) та їхні зв'язки.

3. Проєктування бази даних (ER-Model / Data Schema) Спроєктуйте схему зберігання даних. Вибір типу діаграми залежить від обраної СКБД:

- Варіант А: Реляційна база даних (SQL)
 - Побудуйте класичну ER-діаграму (Entity-Relationship Diagram).
 - Вкажіть сутності, атрибути, первинні (PK) та зовнішні (FK) ключі.
 - Визначте типи зв'язків (1:1, 1:N, M:N).
 - Забезпечте нормалізацію даних (мінімум до 3-ї нормальної форми).
- Варіант Б: Нереляційна база даних (NoSQL)
 - Створіть діаграму або схему колекцій/документів.
 - Відобразіть структуру JSON-документів для основних сутностей.
 - Вкажіть стратегію зв'язків: Embedding (вкладення даних) чи Referencing (посилання на ID).
 - Обґрунтуйте вибір стратегії (наприклад, "Використано Embedding для коментарів, щоб отримати пост разом із коментарями за один запит").

Очікуваний результат (Звіт):

1. Таблиця або список обраного тех стеку з обґрунтуванням (1-2 абзаци).
2. Три діаграми C4 (Context, Container, Component) з коротким описом до кожної.
3. Схема бази даних (ER-діаграма або схема документів NoSQL).

Хід роботи

Завдання № 1.

1.1 Таблиця технологічного стеку

Шар системи	Технологія
Back-end	Python / FastAPI
Front-end	React JS
СКБД	SQLite
ORM	SQLAlchemy

1.2 Обґрунтування вибору

Back-end: Python / FastAPI

Обрано Python та фреймворк FastAPI через сукупність технічних та практичних переваг. FastAPI забезпечує автоматичну генерацію інтерактивної документації API (Swagger UI / ReDoc) на основі Python-анотацій типів без додаткових налаштувань, що критично важливо для корпоративного веб-застосунку з рольовою моделлю та великою кількістю ендпоінтів. Фреймворк побудований на асинхронному ядрі (ASGI / Uvicorn), що забезпечує обробку сотень одночасних запитів без блокування потоків — це відповідає нефункціональній вимозі підтримки 300+ одночасних користувачів. Вбудована підтримка Pydantic-схем дозволяє декларативно описувати та автоматично валідувати вхідні та вихідні дані (DTO), а система Dependency Injection спрощує реалізацію шарової архітектури та підключення JWT-авторизації. FastAPI нативно інтегрується з SQLAlchemy, що усуває необхідність додаткових адаптерів для роботи з базою даних.

Front-end: React JS

React обрано як найпоширеніший інструмент для побудови SPA-застосунків корпоративного класу. Компонентна архітектура React ідеально підходить для реалізації складного інтерфейсу системи Taskerito: дошка задач

(Kanban), модальні вікна проєктів, форми призначення виконавців та дашборди прогресу логічно відокремлюються у незалежні reusable-компоненти. Використання React разом із бібліотеками React Router та React Query забезпечує декларативну навігацію між розділами та ефективну синхронізацію стану з сервером без повного перезавантаження сторінки. Рольова модель (Учасник, Менеджер, Адміністратор) природно реалізується через контекст авторизації та умовний рендеринг компонентів залежно від ролі поточного користувача.

СКБД: SQLite + ORM: SQLAlchemy

Обрано SQLite як базу даних через простоту розгортання (зберігається у вигляді одного файлу без необхідності окремого серверного процесу), що відповідає масштабу навчального корпоративного проєкту. SQLAlchemy використовується як ORM-шар, що забезпечує декларативне оголошення моделей даних у вигляді Python-класів, автоматичні міграції схеми через Alembic, захист від SQL-ін'єкцій та можливість безболісного переходу на PostgreSQL або MS SQL у разі зростання навантаження без зміни бізнес-логіки.

Завдання № 2.

2.1 System Context Diagram (Рівень 1)

System Context Diagram відображає систему Taskerito як єдину "чорну скриньку" та показує всіх зовнішніх акторів, що взаємодіють із нею. На цьому рівні не показуються внутрішні технічні деталі — лише межі системи та зовнішня взаємодія.

Актори системи:

- **Учасник проєкту (Project Member)** — взаємодіє із Taskerito через браузер; надсилає запити на перегляд призначених задач, зміну їх статусу та додавання коментарів.
- **Менеджер проєкту (Project Manager)** — взаємодіє із Taskerito через браузер; надсилає запити на створення проєктів, управління задачами, призначення виконавців та перегляд дашборду прогресу.

- **Адміністратор (Administrator)** — взаємодіє із Taskerito через браузер; надсилає запити на управління обліковими записами та ролями користувачів, перегляд статистики системи.

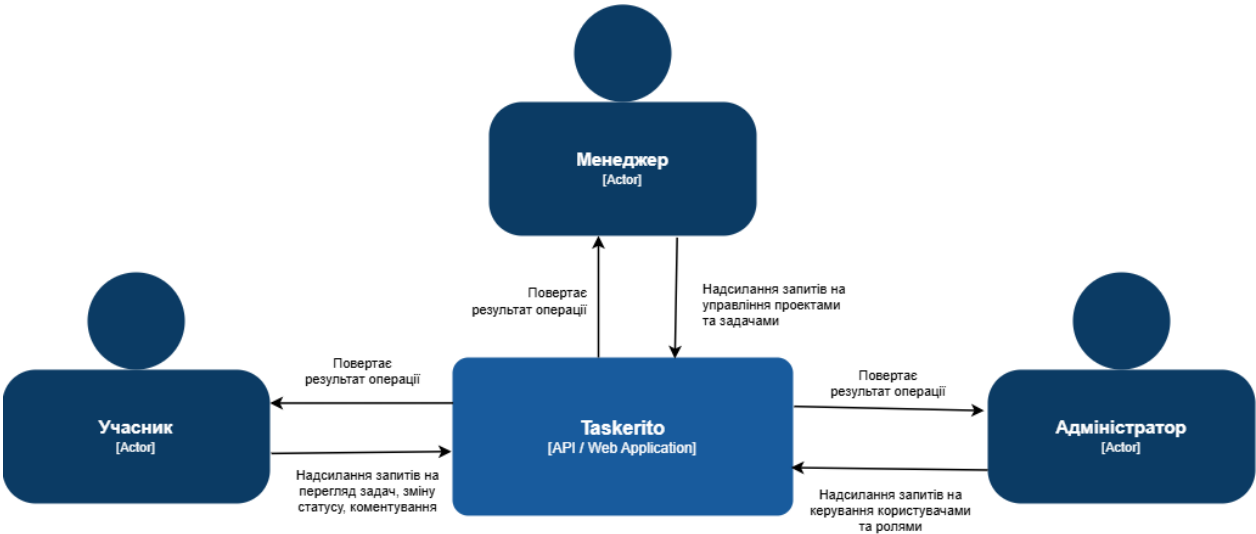


Рисунок 1 — System Context Diagram.

2.2 Container Diagram (Рівень 2)

Container Diagram "відкриває" систему Taskerito та показує її основні технічні блоки (контейнери), їх технології та протоколи взаємодії між ними.

Таблиця контейнерів

Контейнер	Технологія	Призначення
Web Application (SPA)	React JS / Vite / React Router	Відображає інтерфейс користувача у браузері. Надсилає HTTP-запити до REST API. Реалізує рольовий рендеринг компонентів залежно від JWT-токена.
REST API	Python / FastAPI / Uvicorn (ASGI)	Обробляє HTTP-запити від SPA. Реалізує

		аутентифікацію та авторизацію на основі JWT. Виконує бізнес-логіку: управління проектами, задачами та коментарями. Повертає JSON-відповіді.
Database	SQLite (файл db.sqlite3) через SQLAlchemy ORM	Зберігає всі персистентні дані системи: користувачів, ролі, проекти, задачі, коментарі та зв'язки учасників проектів.

Взаємодія між контейнерами

Web Application → REST API

Протокол: HTTPS. Формат даних: JSON. Виконуються запити методів GET, POST, PATCH, DELETE для операцій аутентифікації, управління проектами, задачами, коментарями та користувачами. Отримує відповіді у вигляді JSON-об'єктів та HTTP status codes (200, 201, 400, 401, 403, 404, 422).

REST API → Database

Протокол: SQLAlchemy ORM (SQL-запити через SQLite driver). Виконуються операції SELECT, INSERT, UPDATE, DELETE. Отримує результати запитів у вигляді Python-об'єктів ORM-моделей та підтвердження транзакцій.



Рисунок 2 — Container Diagram.

2.3 Component Diagram (Рівень 3)

Component Diagram деталізує внутрішню архітектуру контейнера REST API (Python / FastAPI). Показано всі компоненти системи, їх відповідальність та зв'язки між ними.

Основні компоненти

1. Routes (API Layer) — маршрутизатори запитів

Відповідають за: прийом HTTP-запитів, первинну валідацію вхідних Pydantic-схем, виклик відповідного сервісу, формування HTTP-відповіді з коректним статус-кодом та серіалізованим DTO.

Компоненти:

- **auth_router** — маршрути реєстрації та авторизації; управляє видачею та оновленням JWT-токенів.
- **users_router** — маршрути управління користувачами; доступний лише для Адміністратора (залежність `verify_admin`).
- **projects_router** — маршрути управління проектами; доступний для Менеджера та Адміністратора.
- **project_members_router** — маршрути управління учасниками проектів; додавання та видалення учасників.
- **tasks_router** — маршрути управління задачами; доступний для всіх ролей з різним рівнем прав.
- **comments_router** — маршрути управління коментарями до задач; доступний для всіх авторизованих користувачів.

- **dashboard_router** — маршрути агрегованої статистики; дашборд прогресу для Менеджера та Адміністратора.

2. Dependencies (Auth & Authorization Layer) — шар залежностей

Реалізує FastAPI Dependency Injection для аутентифікації та авторизації:

- **get_current_user** — декодує JWT-токен з заголовка Authorization Bearer, повертає поточного користувача або HTTP 401.
- **verify_manager** — перевіряє, що роль поточного користувача є Manager або Admin, або повертає HTTP 403.
- **verify_admin** — перевіряє, що роль поточного користувача є Admin, або повертає HTTP 403.
- **get_db** — надає сесію SQLAlchemy з автоматичним закриттям після завершення запиту (yield-залежність).

3. Services (Business Logic Layer) — шар бізнес-логіки

Інкапсулює всі бізнес-правила системи. Сервіси не залежать від HTTP-шару та отримують дані через репозиторії.

- **AuthService** — хешування паролів (bcrypt via passlib), генерація та верифікація JWT-токенів (python-jose), логіка входу та реєстрації.
- **UserService** — управління обліковими записами: отримання списку, блокування/активація, зміна ролі, видалення.
- **ProjectService** — створення проєктів із призначенням автора, оновлення, архівування, перевірка прав доступу (чи є поточний користувач учасником або власником).
- **ProjectMemberService** — додавання та видалення учасників проєкту, перевірка унікальності зв'язку.
- **TaskService** — бізнес-логіка управління задачами: перевірка приналежності задачі до проєкту, дотримання правил переходу статусів (To Do → In Progress → Done / Cancelled), перевірка прав на редагування (лише Менеджер або призначений виконавець), автоматичне визначення прострочених задач.

- **CommentService** — додавання коментарів із прив'язкою до задачі та автора, видалення власних коментарів або будь-яких для Менеджера/Адміна.
- **DashboardService** — агрегація статистики: кількість задач по статусах, список прострочених задач, загальний прогрес проекту у відсотках.

4. Repositories (Data Access Layer) — шар доступу до даних

Відповідають виключно за CRUD-операції з базою даних через SQLAlchemy-сесію. Ізолюють бізнес-логіку від деталей реалізації БД.

- **UserRepository** — *getbyid, getbyemail, get_all* (з фільтрами), *create, update, delete*.
- **RoleRepository** — *getbyname, get_all*.
- **ProjectRepository** — *getbyid, getbyuser* (проекти, до яких має доступ користувач), *create, update, delete*.
- **ProjectMemberRepository** — *getmembersbyproject, addmember, removemember, ismember*.
- **TaskRepository** — *getbyid, getbyproject, getbyassignee, getoverdue, create, update, updatestatus, delete*.
- **CommentRepository** — *getbytask, create, delete*.

5. Models (ORM Layer) — декларативні SQLAlchemy-моделі

- **Role** — ORM-модель таблиці *roles*.
- **User** — ORM-модель таблиці *users* із зовнішнім ключем *role_id*.
- **Project** — ORM-модель таблиці *projects* із зовнішнім ключем *created_by*.
- **ProjectMember** — ORM-модель таблиці *project_members* (асоціативна).
- **Task** — ORM-модель таблиці *tasks* із зовнішніми ключами *projectid, assignedto, created_by*.
- **Comment** — ORM-модель таблиці *comments* із зовнішніми ключами *taskid, userid*.

6. Schemas (Pydantic DTO Layer) — схеми валідації та серіалізації

- **UserCreate / UserRead / UserUpdate** — схеми для операцій з користувачами.
- **LoginRequest / TokenResponse** — схеми аутентифікації.
- **ProjectCreate / ProjectRead / ProjectUpdate** — схеми проєктів.
- **TaskCreate / TaskRead / TaskUpdate / TaskStatusUpdate** — схеми задач.
- **CommentCreate / CommentRead** — схеми коментарів.
- **DashboardStats** — агрегована схема дашборду.

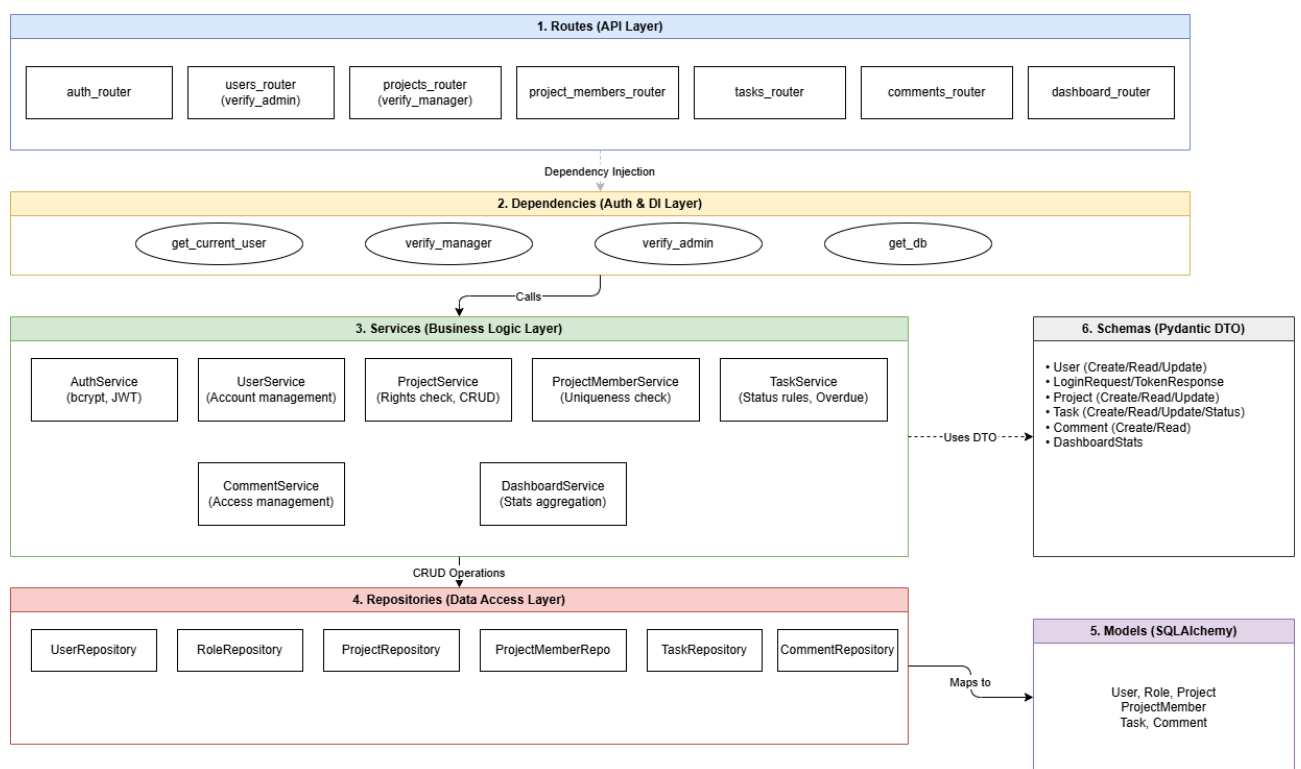


Рисунок 3 — Component Diagram (деталізація контейнера REST API).

Завдання № 3.

Опис структури бази даних Taskerito

Для системи управління проектами та задачами обрано реляційну базу даних SQLite (Варіант А). ER-діаграма проєктується відповідно до третьої нормальної форми (3NF).

1. Таблиця roles

Таблиця призначена для зберігання ролей користувачів у системі та забезпечення рольової моделі доступу.

Поля:

- **id** (PK, INTEGER, AUTOINCREMENT) — унікальний ідентифікатор ролі.
- **name** (VARCHAR(50), UNIQUE, NOT NULL) — назва ролі. Допустимі значення: Member (учасник проекту), Manager (менеджер проекту), Admin (адміністратор системи).

2. Таблиця users

Таблиця містить інформацію про всіх зареєстрованих користувачів системи.

Поля:

- **id** (PK, INTEGER, AUTOINCREMENT) — унікальний ідентифікатор користувача.
- **full_name** (VARCHAR(150), NOT NULL) — повне ім'я користувача для відображення в інтерфейсі.
- **email** (VARCHAR(150), UNIQUE, NOT NULL) — адреса електронної пошти. Є унікальним ідентифікатором для входу в систему.
- **password_hash** (VARCHAR(255), NOT NULL) — хеш пароля користувача, згенерований алгоритмом bcrypt. Оригінальний пароль не зберігається.
- **role_id** (FK → roles.id, INTEGER, NOT NULL) — зовнішній ключ, що визначає роль користувача в системі.

- **is_active** (BOOLEAN, NOT NULL, DEFAULT TRUE) — прапорець активності облікового запису. Заблоковані користувачі не можуть авторизуватися.
- **created_at** (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP) — дата та час реєстрації користувача в системі.

3. Таблиця projects

Таблиця містить інформацію про проекти, які створюються менеджерами для організації задач та команд.

Поля:

- **id** (PK, INTEGER, AUTOINCREMENT) — унікальний ідентифікатор проекту.
- **name** (VARCHAR(200), NOT NULL) — назва проекту.
- **description** (TEXT, NULL) — розширений опис проекту. Необов'язкове поле.
- **status** (VARCHAR(50), NOT NULL, DEFAULT 'Active') — поточний статус проекту. Допустимі значення: Active (активний), Archived (архівований), Completed (завершений).
- **created_by** (FK → users.id, INTEGER, NOT NULL) — зовнішній ключ, що посилається на користувача-менеджера, який створив проект.
- **created_at** (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP) — дата та час створення проекту.

4. Таблиця project_members

Асоціативна таблиця для реалізації зв'язку M:N між проектами та користувачами. Зберігає інформацію про те, які користувачі є учасниками яких проектів.

Поля:

- **id** (PK, INTEGER, AUTOINCREMENT) — унікальний ідентифікатор запису.

- **project_id** (FK → projects.id, INTEGER, NOT NULL) — зовнішній ключ на проект.
- **user_id** (FK → users.id, INTEGER, NOT NULL) — зовнішній ключ на користувача-учасника.
- **joined_at** (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP) — дата та час додавання учасника до проекту.
- **UNIQUE(project_id, user_id)** — складений унікальний індекс, що запобігає дублюванню одного учасника в одному проекті.

5. Таблиця tasks

Таблиця зберігає повну інформацію про задачі в межах проектів. Це центральна сутність системи Taskerito.

Поля:

- **id** (PK, INTEGER, AUTOINCREMENT) — унікальний ідентифікатор задачі.
- **project_id** (FK → projects.id, INTEGER, NOT NULL) — зовнішній ключ на проект, до якого належить задача.
- **title** (VARCHAR(250), NOT NULL) — короткий заголовок задачі.
- **description** (TEXT, NULL) — розширений опис задачі, вимоги або деталі. Необов'язкове поле.
- **status** (VARCHAR(50), NOT NULL, DEFAULT 'To Do') — поточний статус задачі. Допустимі значення: To Do (запланована), In Progress (в роботі), Done (завершена), Cancelled (скасована).
- **priority** (VARCHAR(50), NOT NULL, DEFAULT 'Medium') — пріоритет задачі. Допустимі значення: Low (низький), Medium (середній), High (високий), Critical (критичний).
- **assigned_to** (FK → users.id, INTEGER, NULL) — зовнішній ключ на користувача-виконавця. NULL означає, що задача не призначена.
- **created_by** (FK → users.id, INTEGER, NOT NULL) — зовнішній ключ на користувача (Менеджера), який створив задачу.

- **deadline** (DATETIME, NULL) — граничний термін виконання задачі. NULL означає відсутність дедлайну.
- **created_at** (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP) — дата та час створення задачі.
- **updated_at** (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP) — дата та час останнього оновлення задачі. Оновлюється автоматично ORM при кожній зміні запису.

6. Таблица comments

Таблиця зберігає коментарі користувачів до задач для забезпечення комунікації команди в межах конкретного завдання.

Поля:

- **id** (PK, INTEGER, AUTOINCREMENT) — унікальний ідентифікатор коментаря.
- **task_id** (FK → tasks.id, INTEGER, NOT NULL) — зовнішній ключ на задачу, до якої залишено коментар.
- **user_id** (FK → users.id, INTEGER, NOT NULL) — зовнішній ключ на користувача-автора коментаря.
- **content** (TEXT, NOT NULL) — текстовий зміст коментаря.
- **created_at** (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP) — дата та час створення коментаря.

Опис зв'язків між таблицями

Таблиця 1	Тип зв'язку	Таблиця 2	Опис
roles	1 : N	users	Одна роль може бути призначена багатьом користувачам
users	1 : N	projects	Один користувач (Менеджер) може

			створити багато проектів
projects	M : N	users	Один проект має багатьох учасників; один користувач може бути учасником багатьох проектів (через project_members)
projects	1 : N	tasks	Один проект може містити багато задач
users	1 : N	tasks (assigned_to)	Одному користувачу може бути призначено багато задач
users	1 : N	tasks (created_by)	Один менеджер може створити багато задач
tasks	1 : N	comments	Одна задача може мати багато коментарів
users	1 : N	comments	Один користувач може залишити багато коментарів

Відповідність третій нормальній формі (3NF)

Усі таблиці мають явний первинний ключ (id, AUTOINCREMENT). Усі неключові атрибути кожної таблиці функціонально залежать виключно від первинного ключа: так, атрибут name у таблиці roles залежить лише від roles.id, а не від будь-якого іншого неключового атрибута. Відсутні транзитивні залежності: ім'я ролі не дублюється у таблиці users — зберігається лише зовнішній ключ roleid, що посилається на таблицю roles. Дані не дублюються: інформація про проект зберігається виключно в таблиці projects; задачі посилаються на неї через projectid. Асоціативна таблиця project_members коректно реалізує зв'язок M:N без дублювання даних про проект або користувача.

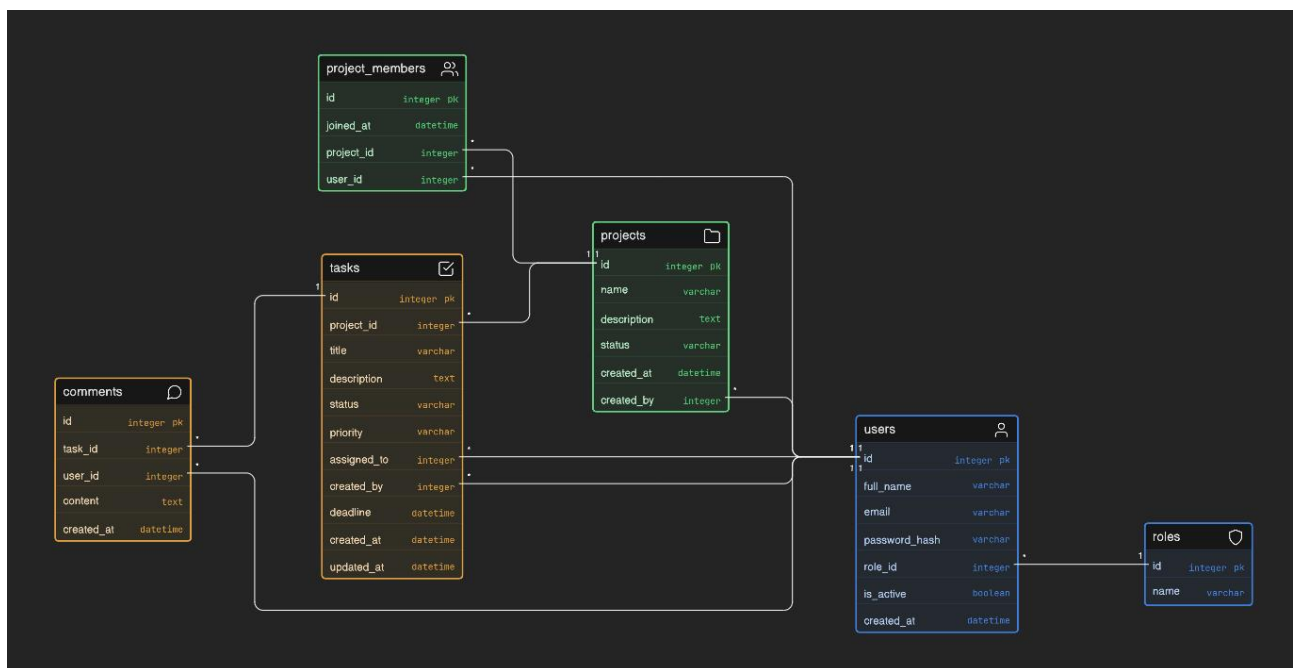


Рисунок 4 — ER-діаграма бази даних Taskerito.